ПРАКТИЧЕСКАЯ РАБОТА №3

Тема работы: разработка и безопасность смарт-контрактов.

Цель работы: спроектировать и реализовать прикладной смарт-контракт для выбранной отрасли, провести функциональное тестирование и тестирование безопасности, выявить уязвимости современными инструментами.

Теоретические основы

Смарт-контракт это программа, которая выполняется виртуальной машиной Ethereum детерминированно: при одинаковом исходном состоянии и одинаковых входных транзакциях все корректные узлы сети должны получить одинаковый результат. Безопасность смарт-контракта зависит не только от правильности программного кода на языке Solidity, но и от архитектуры всей системы: распределения ролей и полномочий, внешних вызовов, управления состоянием, экономических предположений, механизмов обновления контракта и качества тестирования.

Для промышленного использования безопасная разработка должна рассматриваться как непрерывный процесс. Необходимо сочетать безопасное проектирование, использование проверенных библиотек, модульное тестирование, тестирование на случайных входных данных, проверку неизменяемых свойств системы, статический анализ и ручную проверку критически важной логики.

Основные принципы:

- Архитектура должна минимизировать доверие между компонентами. Необходимо заранее определить участников системы, их роли, полномочия, защищаемые активы, допустимые изменения состояния и границы доверия.
- Конфиденциальные данные не следует хранить в открытом виде непосредственно в блокчейне. Для закрытых документов и чувствительной информации обычно сохраняют только криптографический хеш или криптографическое обязательство, а исходные данные размещают во внешнем защищенном хранилище.
- Для сложных систем рекомендуется использовать стандартизированные механизмы управления доступом на основе ролей вместо большого количества самописных проверок адресов. Простая модель единственного владельца допустима для учебных и небольших контрактов, но плохо подходит для систем с большим количеством ролей и административных функций.
- Внешний вызов следует рассматривать как передачу управления потенциально недоверенному программному коду. Для снижения риска повторного входа рекомендуется сначала выполнять проверки, затем изменять внутреннее состояние контракта и только после этого обращаться к внешнему контракту. При необходимости дополнительно используется специальный механизм защиты от повторного входа.
- Тестирование на случайных входных данных позволяет автоматически проверять поведение контракта на большом количестве различных значений. Проверка неизменяемых свойств используется для контроля условий, которые должны оставаться истинными при любой допустимой последовательности операций. Например, пользователь не должен иметь возможность вывести больше средств, чем ему принадлежит, а неавторизованный участник не должен получать административные права.
- Статический анализ помогает автоматически выявлять типовые ошибки: неправильный контроль доступа, опасные внешние вызовы, возможность повторного входа, неиспользуемый код и другие известные дефекты. Однако автоматический анализ не заменяет моделирование угроз, проверку бизнес-логики, ручной анализ программного кода и независимый аудит.

Безопасный смарт-контракт это не только корректный программный код, но и результат правильно спроектированной архитектуры, четкого распределения полномочий, безопасного взаимодействия с внешними компонентами и систематического тестирования.

Проектирование защищенной блокчейн-системы должно выполняться до написания смарт-контрактов, поскольку большинство критических уязвимостей возникает не только из-за ошибок программного кода, но и из-за неправильного распределения полномочий, хранения данных и построения бизнес-процессов.

На первом этапе необходимо определить участников системы, их роли и разрешенные действия. В сложных смарт-контрактах рекомендуется использовать управление доступом на основе ролей, поскольку назначение единственного администратора создает единую точку отказа. Для систем из нескольких контрактов могут применяться готовые механизмы AccessControl и AccessManager библиотеки OpenZeppelin, позволяющие централизованно управлять ролями, разрешенными функциями и задержками выполнения административных операций.

Вторым этапом определяется состояние, которое действительно необходимо хранить непосредственно в блокчейне. Публичная сеть Ethereum не предназначена для хранения конфиденциальных документов в открытом виде. В блокчейн следует помещать идентификаторы, статусы, криптографические хеши, права доступа и события, а исходные документы хранить во внешнем защищенном хранилище. Криптографический хеш позволяет впоследствии доказать, что документ не был изменен.

После определения данных необходимо описать основной поток транзакций и модель угроз. Следует учитывать несанкционированный доступ, подмену данных, повторное использование подписанных сообщений, опережение транзакций, повторный вход при внешних вызовах и компрометацию административного ключа.

Рекомендации по безопасной разработке в Ethereum предполагают явное проектирование контроля доступа, использовать схему «проверки - изменение состояния - внешнее взаимодействие» для защиты от повторного входа и предусматривать механизм аварийного реагирования.

Завершающим этапом являются инварианты безопасности - свойства системы, которые не должны нарушаться ни при какой допустимой последовательности транзакций. Именно они впоследствии становятся основой модульного тестирования, тестирования случайными входными данными и проверки последовательностей операций.

Пример:



https://ethereum.org/developers/docs/nodes-and-clients/?utm_source

Варианты.

Каждый магистрант выполняет работу по назначенному варианту. Вариант определяет предметную область, **набор данных/событий и прикладной сценарий**, технические требования к блокчейн-эксперименту остаются едиными.

№	Отрасль	Прикладной сценарий	Объект исследования / данные
1	Банки и финтех	Проверка статуса межбанковского платёжного поручения	ID операции, сумма, статус, hash документа
2	Страхование	Регистрация этапа обработки страхового случая	claimId, статус, hash доказательства
3	Логистика	Фиксация контрольной точки доставки груза	shipmentId, locationCode, timestamp
4	Здравоохранение	Фиксация согласия пациента на обработку данных	consentId, hash согласия, статус
5	Образование	Проверка цифрового сертификата/диплома	credentialId, hash документа, issuer
6	Энергетика	Регистрация агрегированного показания умного счётчика	meterId, period, reading
7	Нефтегаз	Фиксация состояния технологического оборудования	assetId, stateCode, maintenanceHash
8	Промышленность	Регистрация этапа технического обслуживания	machineId, serviceCode, timestamp
9	Телеком	Регистрация изменения параметра сетевого сервиса	serviceId, configHash, status
10	Ритейл	Фиксация статуса заказа в цепочке поставок	orderId, status, amount
11	Недвижимость	Регистрация этапа согласования сделки	propertyId, stage, documentHash
12	Государственные услуги	Фиксация статуса электронного заявления	requestId, status, agencyCode
13	Транспорт	Регистрация события электронного билета	ticketId, routeCode, validationStatus
14	Туризм	Фиксация статуса бронирования услуги	bookingId, providerId, status
15	Фармацевтика	Регистрация партии лекарственного препарата	batchId, manufacturer, status
16	Кибербезопасность	Фиксация события реагирования на инцидент	incidentId, severity, evidenceHash
17	Облачные сервисы	Регистрация изменения конфигурации ресурса	resourceId, configHash, actor
18	Сельское хозяйство	Фиксация контрольной точки происхождения продукции	lotId, farmId, qualityHash
19	Спорт	Фиксация агрегированного показателя тренировки	athleteIdPseudo, sessionId, metricHash
20	Экология и ESG	Регистрация верифицированного экологического показателя	reportId, period, evidenceHash

Задание 1. Проектирование архитектуры защищённой блокчейн-системы, оценивание 30 баллов / 30 %.

Разработайте архитектуру решения по своему варианту до написания кода. Результат оформляется как одна архитектурная схема и краткая спецификация.

1. **Определите участников системы и их роли**. Укажите, какие категории пользователей будут работать с системой, например: администратор, оператор, аудитор, пользователь. Для каждой роли необходимо описать доступные ей действия и ограничения.
2. **Определите данные, которые будут храниться непосредственно в блокчейне**. Укажите основные сущности системы, их идентификаторы, возможные состояния, события и минимальный набор данных, необходимый для работы смарт-контрактов.
3. **Разделите данные на хранимые в блокчейне и вне блокчейна**. Конфиденциальную информацию, документы и персональные данные не следует размещать в блокчейне в открытом виде. Вместо этого в блокчейне необходимо хранить криптографический хеш или иное подтверждение целостности данных, а исходную информацию размещать во внешнем защищённом хранилище.
4. **Постройте последовательность выполнения основного бизнес-процесса**. Покажите, как пользователь инициирует операцию, какие компоненты системы участвуют в её обработке, какие проверки выполняются и какие изменения происходят в блокчейне.
5. **Разработайте модель угроз.** Необходимо определить не менее пяти возможных угроз безопасности и указать способы защиты от них. Например: несанкционированный доступ, повторный вход в смарт-контракт, подмена данных, повторное использование транзакции, опережение транзакции, злоупотребление бизнес-логикой, компрометация административного ключа.
6. **Определите не менее трёх инвариантов безопасности.** Инвариант - это правило, которое должно выполняться всегда независимо от последовательности транзакций. Например: пользователь не может получить доступ к функции без соответствующей роли; сумма выведенных средств не может превышать сумму внесённых; отозванный или аннулированный объект не может быть повторно активирован без предусмотренной процедуры.

Итог работы должен включать: архитектурную схему системы, описание ролей, структуру хранимых данных, основной поток транзакций, модель угроз и перечень инвариантов безопасности

Задание 2. Реализация и тестирование смарт-контракта, оценивание 35 баллов / 35 %.

1. **Создайте проект по варианту предметной области:** для чётных вариантов - Foundry, для нечётных - Hardhat 3.x. Реализуйте смарт-контракт, содержащий не менее 4 публичных или внешних функций, 2 событий и 2 ролей пользователей**.**
2. **Используйте** пользовательские ошибки, модификаторы или ролевую модель доступа и проверку входных данных**.** Конфиденциальную информацию не храните в блокчейне в открытом виде.
3. Если контракт выполняет перевод ETH, токенов или внешние вызовы, **обеспечьте защиту от повторного входа и безопасный порядок изменения состояния.**
4. Разработайте не менее **8 модульных тестов**, включая корректные сценарии, ошибки доступа, некорректные входные данные, повторные операции, граничные значения и ожидаемые откаты транзакций.
5. **Добавьте не менее 2 тестов со случайными входными данными** либо 2 тестов инвариантов безопасности**.**
6. **Разверните контракт сначала в локальной сети, затем в Sepolia**. В отчёте укажите адрес контракта, хеш транзакции развертывания и не менее двух хешей транзакций взаимодействия с контрактом**.**

Задание 3. Анализ уязвимости и проверка исправления, оценивание 35 баллов / 35 %.

1. **Создайте отдельную уязвимую версию контракта и реализуйте один контролируемый дефект безопасности.** Эксперимент выполняйте только локально или в тестовой сети.
2. **Выберите одну уязвимость релевантную варианту:** нарушение контроля доступа, повторный вход, небезопасный внешний вызов, неправильное использование tx.origin, отказ в обслуживании, ошибка бизнес-логики или небезопасное обновление контракта.
3. **Проведите статический анализ с помощью** Slither, сохраните отчёт и классифицируйте найденные проблемы как реальные уязвимости, ложные срабатывания или информационные замечания.
4. **Воспроизведите уязвимость с помощью теста или специального атакующего контракта.** Зафиксируйте условия возникновения, последовательность транзакций и полученный результат.
5. **Исправьте уязвимость с использованием безопасных шаблонов проектирования** и проверенных библиотек. Подтвердите тестами, что атака после исправления больше не выполняется.
6. **Повторно запустите модульные, случайные и инвариантные тесты**, сравните результаты до и после исправления и кратко опишите оставшиеся риски, требующие ручного аудита. Для выбранной уязвимости объясните первопричину, нарушенное правило безопасности и условия, необходимые для успешной эксплуатации.

Требования к отчёту

- Архитектурная диаграмма.
- Threat model: минимум 5 угроз и 3 security invariant.
- Исходный код контракта и ссылка/архив проекта; используемая версия Solidity/OpenZeppelin/toolchain.
- Результаты минимум 8 unit tests и 2 fuzz/invariant tests. Для каждого тестового этапа представить команду запуска и результат выполнения. Для Sepolia предоставить ссылки на block explorer. Для Slither приложить полный отчёт и краткую таблицу найденных проблем.
- Slither report: найденные issues, классификация и меры исправления.
- Доказательство локальной эксплуатации выбранного дефекта и повторный тест после исправления.
- Sepolia: адрес исправленного контракта, deployment tx hash и минимум 2 interaction tx hash.
- Вывод: какие меры действительно уменьшают риск, а какие требуют дополнительного аудита.

Рекомендуемые официальные источники и документация

1. OpenZeppelin Contracts 5.x Access. https://docs.openzeppelin.com/contracts/5.x/api/access — AccessControl, AccessManager, Ownable.

2. Foundry Forge / Anvil / Cast. https://getfoundry.sh/ — Build, test, fuzz/invariant, deploy, local node.

3. Foundry Invariant Testing. https://getfoundry.sh/forge/invariant-testing — Property/invariant-based testing.

4. Slither. https://github.com/crytic/slither Статический анализ Solidity/Vyper.

5. Echidna. https://github.com/crytic/echidna Property-based fuzzing для Ethereum smart contracts.

6. Hardhat 3. https://hardhat.org/ Solidity/TypeScript testing и debugging.

7. Ethereum Sepolia/Hoodi. https://ethereum.org/developers/docs/networks/ — Актуальные публичные тестовые сети Ethereum.

- Работа не может получить более 50 баллов, если контракт не компилируется или отсутствуют тесты.
- Работа не может получить более 70 баллов без воспроизведения уязвимости.
- Размещение настоящих закрытых ключей, секретов или персональных данных в репозитории считается критической ошибкой.

Контрольные вопросы.

1. Как должна измениться архитектура смарт-контракта, если компрометация административного ключа рассматривается как реалистичная угроза, и какие механизмы позволяют уменьшить последствия такой компрометации?
2. Почему защита от повторного входа не должна ограничиваться только использованием ReentrancyGuard, и какую роль в этом играет порядок «проверки → изменение состояния → внешний вызов»?
3. Как сформулировать инвариант безопасности для контракта так, чтобы его можно было проверить автоматически с помощью invariant testing, и чем такой инвариант отличается от обычного функционального требования?
4. Какие ограничения имеет статический анализ Slither при поиске ошибок бизнес-логики, и почему отсутствие критических предупреждений в отчёте не является доказательством безопасности контракта?
5. Как можно доказать, что исправление уязвимости действительно устранило причину атаки, а не только заблокировало один конкретный сценарий её воспроизведения?